26-单片机 8

大家晚上好，欢迎参加张飞实战电子 FOC 第 11 期学习。

上节课我们一起学习了 PWM 模块部分，让大家试一下编写互补式 PWM 输出。我们今晚讲 PWM 模块初始化，主要是 lesson2 里面的 sfr.h 内容。如果大家遇到困难，需要和我互动你的难点。那我们一起来捋一下，看看这个中心对齐，带 2us 死区的互补 PWM 该怎么实现啊？主要看下，对应的寄存器的设置。

表 4-12：PWM 寄存器映射（仅限 dsPIC33EPXXXMC20X/50X 和 PIC24EPXXXMC20X 器件）

寄存器名称	地址	Bit 15	Bit 14	Bit 13	Bit 12	Bit 11	Bit 10	Bit 9	Bit 8	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	所有复位时的状态
PTCON	0C00	PTEN	—	PTSIDL	SESTAT	SEIEN	EIPU	SYNCPOL	SYNCOEN	SYNCEN	SYNCSRC<2:0>			SEVTPS<3:0>				0000
PTCON2	0C02	—	—	—	—	—	—	—	—	—	—	—	—	—	PCLKDIV<2:0>			0000
PTPER	0C04	PTPER<15:0>																00F8
SEVTCMP	0C06	SEVTCMP<15:0>																0000
MDC	0C0A	MDC<15:0>																0000
CHOP	0C1A	CHPCLKEN	—	—	—	—	—	CHOPCLK<9:0>										0000
PWMKEY	0C1E	PWMKEY<15:0>																0000

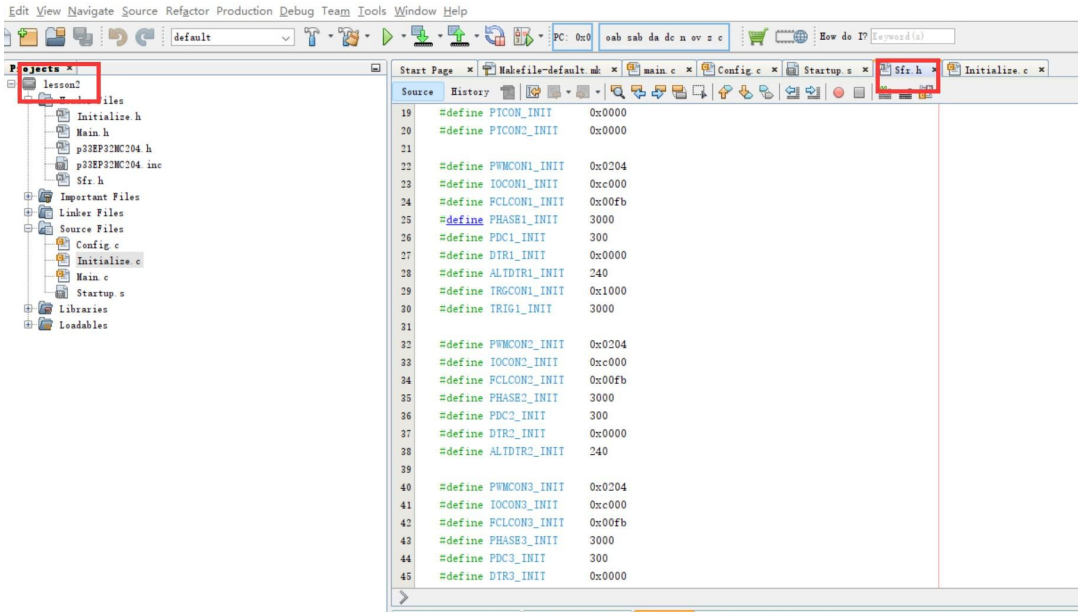
图注：— = 未实现，读为 0。复位值以十六进制显示。

表 4-13：PWM 发生器 1 寄存器映射（仅限 dsPIC33EPXXXMC20X/50X 和 PIC24EPXXXMC20X 器件）

寄存器名称	地址	Bit 15	Bit 14	Bit 13	Bit 12	Bit 11	Bit 10	Bit 9	Bit 8	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	所有复位时的状态
PWMCON1	0C20	FLTSTAT	CLSTAT	TRGSTAT	FLTIEN	CLIEN	TRGIEN	ITB	MDCS	DTC<1:0>		DTCP	—	MTBS	CAM	XPRES	IUE	0000
IOCON1	0C22	PENH	PENL	POLH	POLL	PMOD<1:0>		OVRENH	OVRENL	OVRDAT<1:0>		FLTDAT<1:0>		CLDAT<1:0>		SWAP	OSYNC	C000
FCLCON1	0C24	—	CLSRC<4:0>					CLPOL	CLMOD	FLTSRC<4:0>				FLTPOL	FLTMOD<1:0>			0000
PDC1	0C26	PDC1<15:0>																FFF8
PHASE1	0C28	PHASE1<15:0>																0000
DTR1	0C2A	—	—	DTR1<13:0>														0000
ALTDTR1	0C2C	—	—	ALTDTR1<13:0>														0000
TRIG1	0C32	TRGCMP<15:0>																0000
TRGCON1	0C34	TRGDIV<3:0>				—	—	—	—	—	—	TRGSTRT<5:0>						0000
LEBCON1	0C3A	PHR	PHF	PLR	PLF	FLTLEBEN	CLLEBEN	—	—	—	—	BCH	BCL	BPHH	BPHL	BPLH	BPLL	0000
LEBDLY1	0C3C	—	—	—	—	LEB<11:0>												0000
AUXCON1	0C3E	—	—	—	—	BLANKSEL<3:0>				—	—	CHOPCLK<3:0>				CHOPHEN	CHOPLEN	0000

图注：— = 未实现，读为 0。复位值以十六进制显示。

整个 PWM 模块，包含这些寄存器。但是呢，有的寄存器，我们是不需要用到的。所以呢，有的寄存器也不必去配置。



本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
18
19    #define PTCON_INIT        0x0000
20    #define PTCON2_INIT       0x0000
21
22    #define PWMCON1_INIT      0x0204
23    #define IOCON1_INIT       0xc000
24    #define FCLCON1_INIT      0x0003
25    #define PHASE1_INIT       3000
26    #define PDC1_INIT         300
27    #define DTR1_INIT         0x0000
28    #define ALTDTR1_INIT      240
29    #define TRGCON1_INIT      0x1000
30    #define TRIG1_INIT        3000
```

这个呢，是 sfr.h 文件中，定义的 PWM 模块寄存器初始值。我们顺着来看下，这些寄存器，为什么要配置为这样的值？

16.3 PWMx 控制寄存器

寄存器 16-1： PTCON：PWMx 时基控制寄存器

R/W-0	U-0	R/W-0	HS/HC-0	R/W-0	R/W-0	R/W-0	R/W-0
PTEN	—	PTSIDL	SESTAT	SEIEN	EIPU(1)	SYNCPOL(1)	SYNCOEN(1)
bit 15							bit 8

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
SYNCEN(1)	SYNCSRC<2:0>(1)			SEVTPS<3:0>(1)			
bit 7							bit 0

这个寄存器呢，它是 PWM 模块的时基控制寄存器。

bit 15 **PTEN**：PWMx 模块使能位
1 = 使能 PWMx 模块
0 = 禁止 PWMx 模块

bit 14 **未实现**：读为 0

bit 13 **PTSIDL**：PWMx 时基空闲模式停止位
1 = PWMx 时基在 CPU 空闲模式下暂停
0 = PWMx 时基在 CPU 空闲模式下运行

bit 12 **SESTAT**：特殊事件中断状态位
1 = 特殊事件中断处于待处理状态
0 = 特殊事件中断不处于待处理状态

bit 11 **SEIEN**：特殊事件中断允许位
1 = 允许特殊事件中断
0 = 禁止特殊事件中断

bit 10 **EIPU**：使能立即周期更新位(1)
1 = 立即更新有效周期寄存器
0 = 在 PWMx 周期边界处更新有效周期寄存器

bit 9 **SYNCPOL**：同步输入和输出极性位(1)
1 = SYNCI1/SYNCO1 极性反相（低电平有效）
0 = SYNCI1/SYNCO1 为高电平有效

bit 8 **SYNCOEN**：主时基同步使能位(1)

对于我们的应用来说，可以主要关心红框中的这两个位。bit15，是整个 PWM 模块的开关使能位。这个开关使能位需要注意一下，有的寄存器的配置，需要在模块禁止的时候 配置才会生效 。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

问：开关使能位会有说明吗？

答：这种做法是单片机外设模块配置的标准动作。比如定时器模块在设置的时候，通常关闭使能；而在设置完成后才会打开使能。

所以，整个模块的使能位会在寄存器初始值设置完毕后，设置为 1。然后，bit10 的话，一般也是默认是 0，寄存器都是在 PWM 周期边界处更新。在单电阻的应用场景下，一个周期会触发两次采样，所以，会用到立即更新标志。而其它的位，基本保持默认值 0 即可。所以，PTCON 这个寄存器，初始值就设置为 0 即可。

```
18
19  #define PTCON_INIT      0x0000
20  #define PTCON2_INIT     0x0000
21
22  #define PWMCON1_INIT    0x0204
23  #define IOCON1_INIT     0xc000
24  #define FCLCON1_INIT    0x0003
```

接着，我们来看下 PTCON2 寄存器。

寄存器 16-2：　PTCON2：PWMx 主主控时钟分频比选择寄存器

U-0	U-0	U-0	U-0	U-0	U-0	U-0	U-0
—	—	—	—	—	—	—	—
bit 15							bit 8

U-0	U-0	U-0	U-0	U-0	R/W-0	R/W-0	R/W-0
—	—	—	—	—	PCLKDIV<2:0>(1)		
bit 7							bit 0

这个寄存器，就是设置 PWM 模块的工作时钟频率的。

bit 15-3　**未实现：**读为 0

bit 2-0　**PCLKDIV<2:0>：**PWMx 输入时钟预分频比选择位 (1)

111 = 保留
110 = 64 分频
101 = 32 分频
100 = 16 分频
011 = 8 分频
010 = 4 分频
001 = 2 分频
000 = 1 分频，最大 PWMx 时序分辨率（上电默认值）

它默认呢，是对 Fosc 的 1 分频。也就是说，默认的工作频率是 Fosc=120MHz. 一般呢，PWM 模块都是工作在最高频率，这样 PWM 的分辨率最高。所以，时钟的分频寄存器呢，也就设置为 1 分频。也就是模块工作在 120MHz 频率下。

寄存器 16-2：　PTCON2：PWMx 主主控时钟分频比选择寄存器

U-0	U-0	U-0	U-0	U-0	U-0	U-0	U-0
—	—	—	—	—	—	—	—
bit 15							bit 8

U-0	U-0	U-0	U-0	U-0	R/W-0	R/W-0	R/W-0
—	—	—	—	—	PCLKDIV<2:0>(1)		
bit 7							bit 0

图注：
R = 可读位　W = 可写位　U = 未实现位，读为 0
-n = POR 时的值　1 = 置 1　0 = 清零　x = 未知

bit 15-3　**未实现：**读为 0

bit 2-0　**PCLKDIV<2:0>：**PWMx 输入时钟预分频比选择位 (1)

111 = 保留
110 = 64 分频
101 = 32 分频
100 = 16 分频
011 = 8 分频
010 = 4 分频
001 = 2 分频
000 = 1 分频，最大 PWMx 时序分辨率（上电默认值）

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

所以，这个寄存器就设置为 0 了

```
19  #define PTCON_INIT      0x0000
20  #define PTCON2_INIT     0x0000
```

问：pwm 不是说 20KHZ 吗？怎么这里是 120MHZ？
答：这是 PWM 模块的输入时钟频率，不是 PWM 信号的输出频率。

PWMCONx 这个寄存器，就是设置 PWM 模块的一些细节功能了，我们一起来了解一下

寄存器 16-7：　PWMCONx：PWMx 控制寄存器

HS/HC-0	HS/HC-0	HS/HC-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
FLTSTAT(1)	CLSTAT(1)	TRGSTAT	FLTIEN	CLIEN	TRGIEN	ITB(2)	MDCS(2)
bit 15							bit 8

R/W-0	R/W-0	R/W-0	U-0	R/W-0	R/W-0	R/W-0	R/W-0
DTC<1:0>		DTCP(3)	—	MTBS	CAM(2,4)	XPRES(5)	IUE(2)
bit 7							bit 0

bit 15　**FLTSTAT**：故障中断状态位 (1)
1 = 故障中断处于待处理状态
0 = 没有故障中断处于待处理状态
该位通过设置 FLTIEN = 0 进行清零。

bit 14　**CLSTAT**：限流中断状态位 (1)
1 = 限流中断处于待处理状态
0 = 没有限流中断处于待处理状态
该位通过设置 CLIEN = 0 进行清零。

bit 13　**TRGSTAT**：触发中断状态位
1 = 触发中断处于待处理状态
0 = 没有触发中断处于待处理状态
该位通过设置 TRGIEN = 0 进行清零。

bit 12　**FLTIEN**：故障中断允许位
1 = 允许故障中断
0 = 禁止故障中断，并且 FLTSTAT 位清零

bit15~bit12 这四个位，都是跟故障或限流功能有关的。bit15~bit13 都是中断状态位。通过这些位，我们可以知道是否有故障发生。当然，这些状态位变化，需要在对应的中断使能后才知道。然后，bit12 是故障中断允许位，需要故障中断，就设置为 1 即可。

bit 11　**CLIEN**：限流中断允许位
1 = 允许限流中断
0 = 禁止限流中断，并且 CLSTAT 位清零

bit 10　**TRGIEN**：触发中断允许位
1 = 触发事件产生中断请求
0 = 禁止触发事件中断，并且 TRGSTAT 位清零

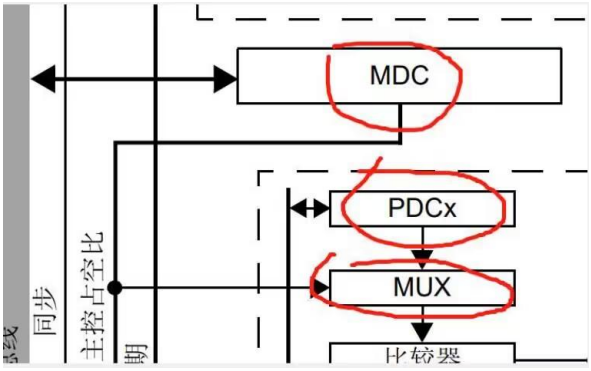
bit 9　**ITB**：独立时基模式位 (2)
1 = PHASEx 寄存器为该 PWM 发生器提供时基周期
0 = PTPER 为该 PWM 发生器提供时序

bit 8　**MDCS**：主控占空比寄存器选择位 (2)
1 = MDC 寄存器为该 PWM 发生器提供占空比信息
0 = PDCx 寄存器为该 PWM 发生器提供占空比信息

然后，bit11 跟 bit10 都是中断的允许位，接着，bit9 是选择工作时基的，独立时基的话，也就是 PHASEx 寄存器作为周期寄存器来使用。我们的程序使用独立时基产生 PWM,所以，这个 bit9，我们需要配置为 1。 bit9 等于 0，就会配置为上节讲的主控时基模式工作，PTPER 寄存器控制 PWM 周期。

问：PTPER 为该 PWM 发生器提供时序怎么理解？
答：上节内容有仔细讲解，就是提供同步信号。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



然后，bit8 就是之前框图的 MUX 选择配置。bit8 设置为 1，MUX 就选择 MDC 作为比较值寄存器，bit8 设置为 0，MUX 就选择 PDCx 作为比较值寄存器。我们里选择 PDCx 作为比较器寄存器。

问：BIT12\11\10 初始值为什么都是禁止呢，，故障中断、限流中断我们这个程序都用不到吗？

答:这里只是初始化配置，电机启动时还可以根据需要修改。

问：那这里就可以随便设置吗？

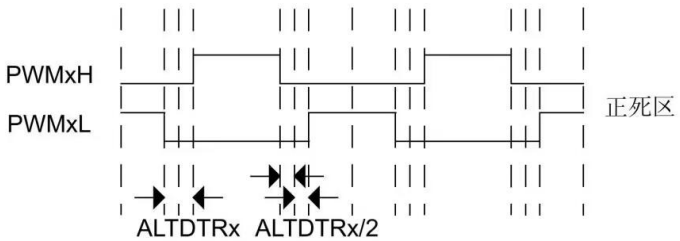
答:至少要让 PWM 模块工作起来，因为我们还需要使用 PWM 模块来触发 ADC 采样。

问：独立时基就必须是 PDCX 吗？

答:是的。

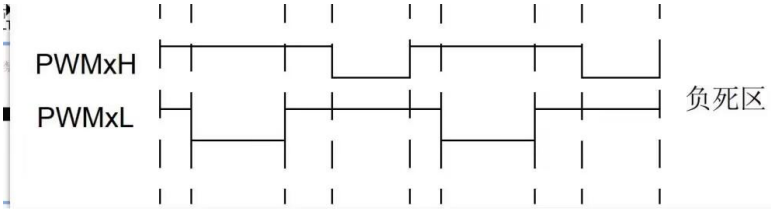
bit 7-6	**DTC<1:0>：** 死区控制位 11 = 死区补偿模式 10 = 禁止死区功能 01 = 对于互补输出模式施加负死区 00 = 对于所有输出模式施加正死区
bit 5	**DTCP：** 死区补偿极性位 (3) 当设置为 1 时： 如果 DTCMPx = 0，则缩短 PWMxL，延长 PWMxH。 如果 DTCMPx = 1，则缩短 PWMxH，延长 PWMxL。 当设置为 0 时： 如果 DTCMPx = 0，则缩短 PWMxH，延长 PWMxL。 如果 DTCMPx = 1，则缩短 PWMxL，延长 PWMxH。
bit 4	**未实现：** 读为 0

然后，bit7~bit6 是设置死区的。这里，有一个正负死区的概念。



正死区，就是我们说过的死区。也就是，在两个沿变化的时候，都同时插入一段为低电平的时间。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



负死区，就是插入同时为高的时间。而中心对齐互补的 PWM 波，不支持插入负死区。只能插入正死区。

bit 7-6	**DTC<1:0>：**死区控制位 11 = 死区补偿模式 10 = 禁止死区功能 01 = 对于互补输出模式施加负死区 00 = 对于所有输出模式施加正死区
bit 5	**DTCP：**死区补偿极性位 (3) 当设置为 1 时： 如果 DTCMPx = 0，则缩短 PWMxL，延长 PWMxH。 如果 DTCMPx = 1，则缩短 PWMxH，延长 PWMxL。 当设置为 0 时： 如果 DTCMPx = 0，则缩短 PWMxH，延长 PWMxL。 如果 DTCMPx = 1，则缩短 PWMxL，延长 PWMxH。
bit 4	**未实现：**读为 0

所以啊，我们这里 bit7~bit6 只能设置为 00。

bit 7-6	**DTC<1:0>：**死区控制位 11 = 死区补偿模式 10 = 禁止死区功能 01 = 对于互补输出模式施加负死区 00 = 对于所有输出模式施加正死区
bit 5	**DTCP：**死区补偿极性位 (3) 当设置为 1 时： 如果 DTCMPx = 0，则缩短 PWMxL，延长 PWMxH。 如果 DTCMPx = 1，则缩短 PWMxH，延长 PWMxL。 当设置为 0 时： 如果 DTCMPx = 0，则缩短 PWMxH，延长 PWMxL。 如果 DTCMPx = 1，则缩短 PWMxL，延长 PWMxH。
bit 4	**未实现：**读为 0

然后，这个死区补偿极性位，是需要配合外部硬件的，我们这里没有用到。

bit 3	**MTBS：**主控时基选择位 1 = PWM 发生器使用辅助主控时基进行同步，并使用它作为 PWM 发生逻辑的时钟源（如果辅助时基可用） 0 = PWM 发生器使用主主控时基进行同步，并使用它作为 PWM 发生逻辑的时钟源
bit 2	**CAM：**中心对齐模式使能位 (2,4) 1 = 使能中心对齐模式 0 = 使能边沿对齐模式
bit 1	**XPRES：**外部 PWMx 复位控制位 (5) 1 = 如果 PWM 发生器处于独立时基模式，则限流源复位该 PWM 发生器的时基 0 = 外部引脚不影响 PWMx 时基
bit 0	**IUE：**立即更新使能位 (2) 1 = 立即更新有效 MDC/PDCx/DTx/ALTDTRx/PHASEx 寄存器 0 = 更新有效 MDC/PDCx/DTx/ALTDTRx/PHASEx 寄存器与 PWMx 周期边界同步

然后，bit3 的话，也是一个时基选择。对于我们单片机来说，只能选择主主控时基所以，必定设置为 0。然后，bit2 是设置 PWM 的模式的。我们要选择中心对齐模式。所以，这个位设置为 1。

问：不是说用独立模式吗?这里为什么只能设置主控模式?

答：这是关于主控时基模式下的时钟源选择配置，这里有两个问题，一是我们配置 PWM 模块工作在独立时基模式下，理论上主控时基的时钟源设置并不关心；二是我们使用的这个型号单片机是这个系列中功能减配的单片机，辅助主控时基不可用。 所以这个 bit3 就设置为 0 了。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

问：如果设置为 1 了，会有什么后果？
答：只是不符合逻辑，应该没有什么太大的后果。

bit1，是一个外部信号复位时基控制位，一般我们也不使用，不关心，默认为 0 即可。接着，bit0，就是之前我们提到过的立即更新使能位。这里，就介绍了会立即更新哪些寄存器，也就是说，设置了这个位为 1 之后，介绍中提到的这些寄存器，只要设置了新的值后，都会立即生效。这个位的话，单电阻应用中，是比较有用的。

```
22  #define PWMCON1_INIT     0x0204
23  #define IOCON1_INIT      0xc000
24  #define FCLCON1_INIT     0x0003
25  #define PHASE1_INIT      3000
26  #define PDC1_INIT        300
27  #define DTR1_INIT        0x0000
28  #define ALTDTR1_INIT     240
29  #define TRGCON1_INIT     0x1000
30  #define TRIG1_INIT       3000
```

所以，根据我们的介绍，这个寄存器的初始设定值就是 0x0204 了。接着，我们来看 IOCON1 寄存器。

寄存器 16-13：　IOCONx：PWMx I/O 控制寄存器 (2)

R/W-1	R/W-1	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
PENH	PENL	POLH	POLL	PMOD<1:0>(1)		OVRENH	OVRENL
bit 15							bit 8

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
OVRDAT<1:0>		FLTDAT<1:0>		CLDAT<1:0>		SWAP	OSYNC
bit 7							bit 0

这个寄存器呢，就是设置 PWM 引脚相关的。

bit 15　**PENH**：PWMxH 输出引脚所有权位
1 = PWMx 模块控制 PWMxH 引脚
0 = GPIO 模块控制 PWMxH 引脚

bit 14　**PENL**：PWMxL 输出引脚所有权位
1 = PWMx 模块控制 PWMxL 引脚
0 = GPIO 模块控制 PWMxL 引脚

bit 13　**POLH**：PWMxH 输出引脚极性位
1 = PWMxH 引脚为低电平有效
0 = PWMxH 引脚为高电平有效

bit 12　**POLL**：PWMxL 输出引脚极性位
1 = PWMxL 引脚为低电平有效
0 = PWMxL 引脚为高电平有效

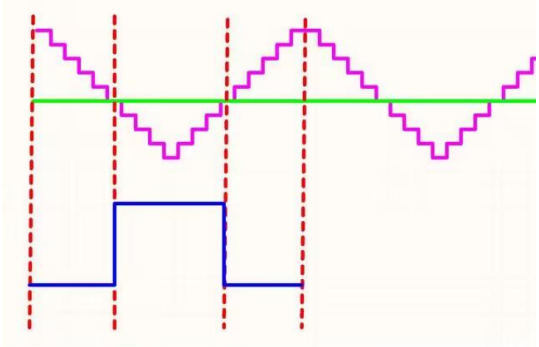
bit15 跟 bit14 是设置两个引脚由什么模块来控制。可以选择是 PWM 模块控制，或者由 GPIO 模块来控制。选择 GPIO 模块控制的话，这两个引脚的输出，就可以由 LATx 寄存器来设置输出高电平，或者低电平。选择 PWM 模块控制的话，这两个引脚的输出，就是靠 PWM 模块控制输出 PWM 波，或者高低电平。一般使用 PWM 模块的话，对应的引脚就是选择 PWM 模块来自己控制，所以，这两个位就设置为 1。然后，bit12 跟 bit13 是设置 PWM 引脚的输出极性。

那有效是什么意思呢？

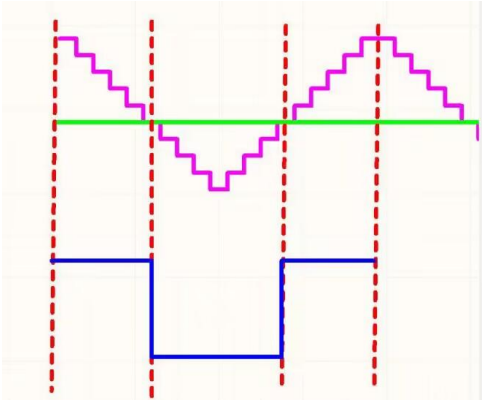
本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

bit 15	**PENH**：PWMxH 输出引脚所有权位 1 = PWMx 模块控制 PWMxH 引脚 0 = GPIO 模块控制 PWMxH 引脚
bit 14	**PENL**：PWMxL 输出引脚所有权位 1 = PWMx 模块控制 PWMxL 引脚 0 = GPIO 模块控制 PWMxL 引脚
bit 13	**POLH**：PWMxH 输出引脚极性位 1 = PWMxH 引脚为低电平有效 0 = PWMxH 引脚为高电平有效
bit 12	**POLL**：PWMxL 输出引脚极性位 1 = PWMxL 引脚为低电平有效 0 = PWMxL 引脚为高电平有效

高电平有效的意思就是，当计数值小于比较值的时候，比较器的输出是高电平。低电平有效的意思就是，当计数值小于比较值的时候，比较器的输出就是低电平。也可以理解为这个极性位可以将输出极性取反。



大家看，这就是设定有效值为高的情况。水红色是计数值，绿色是比较值。当计数值小于比较值，输出就是高电平。



如果有效值为低的话，就是这样的波形。那么，正常情况下，我们都会选择有效值为高的情况。

问：你是从哪里知道这个逻辑的?手册上有说明吗?
答：任何手册不可能有百分百的解释，有的答案是根据仿真和开发经验摸索出来的。

问：应该不是所有的单片机都是这样的把?
答：不同的单片机不一样，都需要仔细阅读手册，然后仿真调试去验证你的理解。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

bit 15 **PENH**：PWMxH 输出引脚所有权位
1 = PWMx 模块控制 PWMxH 引脚
0 = GPIO 模块控制 PWMxH 引脚
bit 14 **PENL**：PWMxL 输出引脚所有权位
1 = PWMx 模块控制 PWMxL 引脚
0 = GPIO 模块控制 PWMxL 引脚
bit 13 **POLH**：PWMxH 输出引脚极性位
1 = PWMxH 引脚为低电平有效
0 = PWMxH 引脚为高电平有效
bit 12 **POLL**：PWMxL 输出引脚极性位
1 = PWMxL 引脚为低电平有效
0 = PWMxL 引脚为高电平有效

所以，这两个位，一般就是设置为 0。

bit 11-10 **PMOD<1:0>**：PWMx # I/O 引脚模式位 (1)
11 = 保留；不要使用
10 = PWMx I/O 引脚对处于推挽输出模式
01 = PWMx I/O 引脚对处于冗余输出模式
00 = PWMx I/O 引脚对处于互补输出模式
bit 9 **OVRENH**：PWMxH 引脚改写使能位
1 = OVRDAT<1> 控制 PWMxH 引脚上的输出
0 = PWMx 发生器控制 PWMxH 引脚
bit 8 **OVRENL**：PWMxL 引脚改写使能位
1 = OVRDAT<0> 控制 PWMxL 引脚上的输出
0 = PWMx 发生器控制 PWMxL 引脚

bit11~bit10，是设置引脚的模式。是互补的，还是其它的。我们这里呢，就是设置互补输出模式。然后，bit9 跟 bit8，就是让引脚强制输出还是非强制输出。如果是正常的 PWM 输出，就是设置为 0.　如果需要强制输出，就设置为 1。那么，具体强制输出的是低电平还是高电平，就需要根据实际需要再配置。

bit 9 **OVRENH**：PWMxH 引脚改写使能位
1 = OVRDAT<1> 控制 PWMxH 引脚上的输出
0 = PWMx 发生器控制 PWMxH 引脚
bit 8 **OVRENL**：PWMxL 引脚改写使能位
1 = OVRDAT<0> 控制 PWMxL 引脚上的输出
0 = PWMx 发生器控制 PWMxL 引脚
bit 7-6 **OVRDAT<1:0>**：使能改写时 PWMxH 和 PWMxL 引脚数据位
如果 OVRENH = 1，则 PWMxH 被驱动为由 OVRDAT<1> 指定的状态。
如果 OVRENL = 1，则 PWMxL 被驱动为由 OVRDAT<0> 指定的状态。

配置的话，就由 bit7　bit6 决定，强制输出高电平，就是 1，强制输出低电平，就是 0。

问：引脚改写使能位是用来干什么用的？
答：比如，故障时关闭 PWM 强制输出无效，开始自举电容充电时，上管关闭，下管 PWM 这种控制需求就可以使用引脚改写输出。

bit 5-4 **FLTDAT<1:0>**：使能 FLTMOD 时 PWMxH 和 PWMxL 引脚数据位
如果故障有效，则 PWMxH 被驱动为由 FLTDAT<1> 指定的状态。
如果故障有效，则 PWMxL 被驱动为由 FLTDAT<0> 指定的状态。
bit 3-2 **CLDAT<1:0>**：使能 CLMOD 时 PWMxH 和 PWMxL 引脚数据位
如果限流有效，则 PWMxH 被驱动为由 CLDAT<1> 指定的状态。
如果限流有效，则 PWMxL 被驱动为由 CLDAT<0> 指定的状态。

然后，这几个位，也是强制让引脚输出　高电平或者低电平。但是，这里的强制是有条件的。也就是在故障发生或者限流发生的时候，强制有效。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

bit 1	**SWAP**：交换 PWMxH 和 PWMxL 引脚位 1 = PWMxH 输出信号连接到 PWMxL 引脚； PWMxL 输出信号连接到 PWMxH 引脚 0 = PWMxH 和 PWMxL 引脚映射到它们各自对应的引脚
bit 0	**OSYNC**：输出改写同步位 1 = 通过 OVRDAT<1:0> 位进行的输出改写与 PWMx 时基同步 0 = 通过 OVDDAT<1:0> 位进行的输出改写在下一个 CPU 时钟边界发生

接着，bit1 是设置引脚交换功能的。可以让 PWMH 跟 PWML 引脚输出波形互换。也就可以方便硬件的走线。最后一个输出改写同步位。bit0，就是设置强制输出的电平是什么时候生效。这个呢，影响很小，一般就是默认为 0。

```
22  #define PWMCON1_INIT     0x0204
23  #define IOCON1_INIT      0xc000
24  #define FCLCON1_INIT     0x0003
25  #define PHASE1_INIT      3000
26  #define PDC1_INIT        300
27  #define DTR1_INIT        0x0000
28  #define ALTDTR1_INIT     240
29  #define TRGCON1_INIT     0x1000
30  #define TRIG1_INIT       3000
```

这样的话，这个寄存器初始值就设置好了。

```
22  #define PWMCON1_INIT     0x0204
23  #define IOCON1_INIT      0xc000
24  #define FCLCON1_INIT     0x0003
25  #define PHASE1_INIT      3000
26  #define PDC1_INIT        300
27  #define DTR1_INIT        0x0000
28  #define ALTDTR1_INIT     240
29  #define TRGCON1_INIT     0x1000
30  #define TRIG1_INIT       3000
```

接着，这个 FCLCONx 寄存器是设定故障或限流模式的。

寄存器 16-15： **FCLCONx：PWMx 故障限流控制寄存器** (1)

U-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
—	CLSRC<4:0>					CLPOL(2)	CLMOD
bit 15							bit 8

R/W-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-0	R/W-0	R/W-0
FLTSRC<4:0>					FLTPOL(2)	FLTMOD<1:0>	
bit 7							bit 0

这个寄存器呢，高 8 位是设置限流，低 8 位是设置故障。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

bit 7-3	**FLTSRC<4:0>**：PWM 发生器 # 的故障控制信号源选择位
	11111 = 故障 32（默认）
	11110 = 保留
	·
	·
	·
	01100 = 保留
	01011 = 比较器 4
	01010 = 运放 / 比较器 3
	01001 = 运放 / 比较器 2
	01000 = 运放 / 比较器 1
	00111 = 保留
	00110 = 保留
	00101 = 保留
	00100 = 保留
	00011 = 故障 4
	00010 = 故障 3
	00001 = 故障 2
	00000 = 故障 1

bit 2	**FLTPOL**：PWM 发生器 # 的故障极性位 (2)
	1 = 选定的故障源为低电平有效
	0 = 选定的故障源为高电平有效

bit 1-0	**FLTMOD<1:0>**：PWM 发生器 # 的故障模式位
	11 = 禁止故障输入
	10 = 保留
	01 = 选定的故障源将 PWMxH 和 PWMxL 引脚强制为 FLTDAT 值（周期）
	00 = 选定的故障源将 PWMxH 和 PWMxL 引脚强制为 FLTDAT 值（锁定状态）

大家看，bit1~0 是设置故障模式的。周期模式的话，就是故障判断条件会每个周期都判断一下。如果当前周期有故障出现，则当前周期会关断 PWM 输出。如果下个周期恢复了，则下个周期的 PWM 会恢复输出。这就是周期模式。如果是锁定模式的话，只要检测到故障，PWM 就会一直关闭。除非人为的去清除故障。清除故障之后，PWM 才会继续恢复。

问：SVPWM 和 FOC 寄存器设置不一样吧?
答：这个问题比较复杂，学习到后面就会知道了。

问：负死区 pwm 输出同时为高，会不会上下管同时导通?
答：会的，所以 FOC 不会使用负死区。

bit 7-3	**FLTSRC<4:0>**：PWM 发生器 # 的故障控制信号源选择位
	11111 = 故障 32（默认）
	11110 = 保留
	·
	·
	·
	01100 = 保留
	01011 = 比较器 4
	01010 = 运放 / 比较器 3
	01001 = 运放 / 比较器 2
	01000 = 运放 / 比较器 1
	00111 = 保留
	00110 = 保留
	00101 = 保留
	00100 = 保留
	00011 = 故障 4
	00010 = 故障 3
	00001 = 故障 2
	00000 = 故障 1

bit 2	**FLTPOL**：PWM 发生器 # 的故障极性位 (2)
	1 = 选定的故障源为低电平有效
	0 = 选定的故障源为高电平有效

bit 1-0	**FLTMOD<1:0>**：PWM 发生器 # 的故障模式位
	11 = 禁止故障输入
	10 = 保留
	01 = 选定的故障源将 PWMxH 和 PWMxL 引脚强制为 FLTDAT 值（周期）
	00 = 选定的故障源将 PWMxH 和 PWMxL 引脚强制为 FLTDAT 值（锁定状态）

然后，bit2 设定故障的极性。可以支持高电平或者低电平，这个呢，就跟我们的硬件有关的。硬件如果是高电平故障有效，则我们这里就要设置极性是高电平。低电平故障有效的话，就要设置极性是低电平。

问：负死区的应用场景是什么?

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

高速 PWM 模块提供正死区和负死区。正死区用于防止 PWM 输出发生重叠。正死区发生可用于所有输出模式。正死区电路的工作方式是对 PWM 信号前沿进行消隐（门控）。负死区用于强制 PWMxH 和 PWMxL 信号发生重叠。负死区的工作方式是延长当前有效 PWM 输出，使延长时间段与刚刚置为有效的 PWM 输出发生重叠。一些转换器技术要求产生一定限度的电流直通。

答：可能是有些高端的数字电源控制。

bit 7-3	**FLTSRC<4:0>**：PWM 发生器 # 的故障控制信号源选择位
	11111 = 故障 32（默认）
	11110 = 保留
	.
	.
	.
	01100 = 保留
	01011 = 比较器 4
	01010 = 运放 / 比较器 3
	01001 = 运放 / 比较器 2
	01000 = 运放 / 比较器 1
	00111 = 保留
	00110 = 保留
	00101 = 保留
	00100 = 保留
	00011 = 故障 4
	00010 = 故障 3
	00001 = 故障 2
	00000 = 故障 1

然后，bit7~bit3 是设置故障的源。



FLT32 是一个默认的故障源。我们的硬件呢，也是接到这个源上的。所以，如果要使能故障功能的话，就要选择这个源。

bit 15	**未实现**：读为 0

bit 14-10	**CLSRC<4:0>**：PWM 发生器 # 的限流控制信号源选择位
	11111 = 故障 32
	11110 = 保留
	.
	.
	.
	01100 = 保留
	01011 = 比较器 4
	01010 = 运放 / 比较器 3
	01001 = 运放 / 比较器 2
	01000 = 运放 / 比较器 1
	00111 = 保留
	00110 = 保留
	00101 = 保留
	00100 = 保留
	00011 = 故障 4
	00010 = 故障 3
	00001 = 故障 2
	00000 = 故障 1（默认）

bit 9	**CLPOL**：PWM 发生器 # 的限流极性位 (2)
	1 = 选定的限流源为低电平有效
	0 = 选定的限流源为高电平有效

bit 8	**CLMOD**：PWM 发生器 # 的限流模式使能位
	1 = 使能限流模式
	0 = 禁止限流模式

然后，限流的设置也类似。但是，它就没有锁定模式了，只有周期模式。因为，限流每个周期的检测。不会一直关闭的。这就是限流跟故障的区别。限流，相当于是某个周期出现了限流有效，则当前周期的 PWM 波被强制拉为低。下个周期，没有限流信号，则 PWM 波要恢复。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

问：感觉这个芯片就是为电机控制定做的一样？
答：是的，电机和数字电源控制。

问：限流信号是指什么信号？
答：课程中没有使用。

问：为什么寄存器的 FCLCONx 中的 X 后面都是 1 啊,为什么不是 0？
答：因为 3 个 PWM 发生器分别是 PWM1 PWM2 PWM3，没有 PWM0。

问：那为什么不设置 PWM2，PWM3，设置了 PWM1 等于 PWM2 和 PWM3 都设置了吗？
答：PWM2 和 PWM3，同 PWM1 的设置是一样的。

```
#define PWMCON1_INIT    0x0204
#define IOCON1_INIT     0xc000
#define FCLCON1_INIT    0x00f3
#define PHASE1_INIT     3000
#define PDC1_INIT       300
#define DTR1_INIT       0x0000
#define ALTDTR1_INIT    240
#define TRGCON1_INIT    0x1000
#define TRIG1_INIT      3000
```

这里呢，我们不使用限流。对应的故障呢，也暂时关闭，故障源选择 FLT32。后面要使用故障的话，就设置对应的模式即可。

```
#define PWMCON1_INIT    0x0204
#define IOCON1_INIT     0xc000
#define FCLCON1_INIT    0x00f3
#define PHASE1_INIT     3000
#define PDC1_INIT       300
#define DTR1_INIT       0x0000
#define ALTDTR1_INIT    240
#define TRGCON1_INIT    0x1000
#define TRIG1_INIT      3000
```

这两个寄存器，一个是周期，一个是占空比。PHASE1 就是周期的设定值，3000 对应的载频是多少呢？

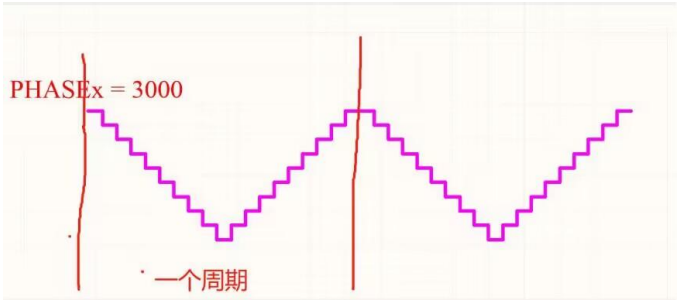
公式 14-2：　　中心对齐模式下的 **PHASEx** 或 **SPHASEx** 寄存器值计算

$$PHASEx,\ SPHASEx = \frac{F_{OSC}\ 120MHz}{F_{PWM} \cdot PWM\ 输入时钟预分频比 \cdot 2}$$

20KHz

大家看下这个公式。中心对齐模式下，周期寄存器的设定值是按照这个公式计算出来的我们设定 PWM 的载频是 20KHz 的话，则 PHASEx 就是 3000。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



因为，中心对齐模式，计数器是下上计数。所以，最大计数值的话，只需要设置一个上坡或者一个下坡的计数值。

```
22  #define PWMCON1_INIT     0x0204
23  #define IOCON1_INIT      0xc000
24  #define FCLCON1_INIT     0x00f3
25  #define PHASE1_INIT      3000
26  #define PDC1_INIT        300
27  #define DTR1_INIT        0x0000
28  #define ALTDTR1_INIT     240
29  #define TRGCON1_INIT     0x1000
30  #define TRIG1_INIT       3000
31
```

然后呢， PDCx 寄存器，就是设置比较值。也就是设置占空比值。初始呢，设置个 10%左右

```
22  #define PWMCON1_INIT     0x0204
23  #define IOCON1_INIT      0xc000
24  #define FCLCON1_INIT     0x00fb
25  #define PHASE1_INIT      3000
26  #define PDC1_INIT        300
27  #define DTR1_INIT        0x0000
28  #define ALTDTR1_INIT     240
29  #define TRGCON1_INIT     0x1000
30  #define TRIG1_INIT       3000
```

接着呢，这两个寄存器是设置死区的。但是呢，在中心对齐模式的互补 PWM 模式下，死区完全由 ALTDTRx 寄存器设置的。DTRx 寄存器，就设置为 0 即可。

公式 14-7： 死区计算

$$ALTDTRx,\ DTRx = FOSC * \frac{所需的死区}{PWM\ 输入时钟预分频比}$$

死区设定寄存器的设定值，可以参考这个公式。假设我们设置 2us 的死区，那么，就可以把 ALTDTRx 寄存器设定值计算出来。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

公式 14-7： 死区计算

$$ALTDTRx, DTRx = FOSC * \frac{所需的死区\ 2us}{PWM\ 输入时钟预分频比} \quad 120MHz$$

2us 计算出来，ALTDTRx = 240。

```
22  #define PWMCON1_INIT     0x0204
23  #define IOCON1_INIT      0xc000
24  #define FCLCON1_INIT     0x00fb
25  #define PHASE1_INIT      3000
26  #define PDC1_INIT        300
27  #define DTR1_INIT        0x0000
28  #define ALTDTR1_INIT     240
29  #define TRGCON1_INIT     0x1000
30  #define TRIG1_INIT       3000
```

最后两个寄存器，是设置触发的。

寄存器 16-12： TRGCONx：PWMx 触发控制寄存器

R/W-0	R/W-0	R/W-0	R/W-0	U-0	U-0	U-0	U-0
TRGDIV<3:0>				—	—	—	—
bit 15							bit 8

U-0	U-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
—	—	TRGSTRT<5:0>(1)					
bit 7							bit 0

图注：			
R = 可读位	W = 可写位	U = 未实现位，读为 0	
-n = POR 时的值	1 = 置 1	0 = 清零	x = 未知

bit 15-12 **TRGDIV<3:0>：** 触发器 # 输出分频比位
1111 = 每发生 16 个触发事件时触发输出
1110 = 每发生 15 个触发事件时触发输出
1101 = 每发生 14 个触发事件时触发输出
1100 = 每发生 13 个触发事件时触发输出
1011 = 每发生 12 个触发事件时触发输出
1010 = 每发生 11 个触发事件时触发输出
1001 = 每发生 10 个触发事件时触发输出
1000 = 每发生 9 个触发事件时触发输出
0111 = 每发生 8 个触发事件时触发输出
0110 = 每发生 7 个触发事件时触发输出
0101 = 每发生 6 个触发事件时触发输出
0100 = 每发生 5 个触发事件时触发输出
0011 = 每发生 4 个触发事件时触发输出
0010 = 每发生 3 个触发事件时触发输出
0001 = 每发生 2 个触发事件时触发输出
0000 = 每次发生触发事件时触发输出

bit 11-6 **未实现：** 读为 0

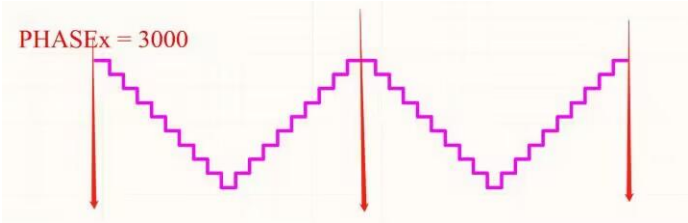
bit 5-0 **TRGSTRT<5:0>：** 触发后分频比开始使能选择位 (1)
111111 = 使能模块之后，在产生第一个触发事件之前先等待 63 个 PWM 周期
.
.
.
000010 = 使能模块之后，在产生第一个触发事件之前先等待 2 个 PWM 周期
000001 = 使能模块之后，在产生第一个触发事件之前先等待 1 个 PWM 周期
000000 = 使能模块之后，在产生第一个触发事件之前先等待 0 个 PWM 周期

触发信号不需要延时，所以，bit5~0 就设置为 0。然后呢，触发信号的分频，我们希望一个周期触发一次。所以，就设置为每 2 个触发事件才会有一个触发信号输出。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
#define PWMCON1_INIT    0x0204
#define IOCON1_INIT     0xc000
#define FCLCON1_INIT    0x00fb
#define PHASE1_INIT     3000
#define PDC1_INIT       300
#define DTR1_INIT       0x0000
#define ALTDTR1_INIT    240
#define TRGCON1_INIT    0x1000
#define TRIG1_INIT      3000
```

然后，最后一个寄存器是设置触发点的。这里大家要注意一下，手册中对于边沿对齐和中心对齐模式下的触发信号分别有详细描述，在 0 和最大值位置触发会有点特殊，具体可以看手册。



根据我们设定的值，触发点设置为 3000。就是在三角波的最高点触发 AD 采样。

```
#define PWMCON2_INIT    0x0204
#define IOCON2_INIT     0xc000
#define FCLCON2_INIT    0x00fb
#define PHASE2_INIT     3000
#define PDC2_INIT       300
#define DTR2_INIT       0x0000
#define ALTDTR2_INIT    240

#define PWMCON3_INIT    0x0204
#define IOCON3_INIT     0xc000
#define FCLCON3_INIT    0x00fb
#define PHASE3_INIT     3000
#define PDC3_INIT       300
#define DTR3_INIT       0x0000
#define ALTDTR3_INIT    240
```

那么，PWM 发生器 2 跟 3 配置跟 PWM 发生器 1 是一样的。

问：如果占空比没有全开的时候是不是在占空比为低电平的时刻采样啊?
答：触发时刻和占空比无关，触发时刻是独立的 TRIG 寄存器设置的。

问：什么时候用 DTR1 设置死区？
答：PWM 边沿对齐模式。

问：采集 ADC 应该希望是有电流的时刻去采集吧？
答：是需要考虑有电流的时刻采样的。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

问：采集的时候是不是应该在 PWM_ON 的时候？
答：不一定，可以是 ON，也可以是 OFF，OFF 的话可以采集续流电流。

问：采集时刻那为什么不关联 PWM_DUTY?
答：可以关联也可以不关联，这是两个独立的寄存器 PDC 和 TRIG 控制的。

问：这里的 0001 是每两次触发 1 次，0010 是每三次触发两次，这样理解吗？
答：是的。

```
11  /*=====================================================================
12  void Initialize(void)
13  {
14      TRISA = TRISA_INIT;
15      LATA = LATA_INIT;
16      ANSELA = ANSELA_INIT;
17
18      PTCONbits.PTEN = 0;//禁止PWM模块
19      PTCON = PTCON_INIT;
20      PTCON2 = PTCON2_INIT;
21      //PWM发生器1的寄存器初始化
22      PWMCON1 = PWMCON1_INIT;
23      IOCON1 = IOCON1_INIT;
24      FCLCON1 = FCLCON1_INIT;
25      PHASE1 = PHASE1_INIT;
26      PDC1 = PDC1_INIT;
27      DTR1 = DTR1_INIT;
28      ALTDTR1 = ALTDTR1_INIT;
29      TRGCON1 = TRGCON1_INIT;
30      TRIG1 = TRIG1_INIT;
31      //PWM发生器2的寄存器初始化
32      PWMCON2 = PWMCON2_INIT;
33      IOCON2 = IOCON2_INIT;
34      FCLCON2 = FCLCON2_INIT;
35      PHASE2 = PHASE2_INIT;
36      PDC2 = PDC2_INIT;
```

最后，大家自己把初始值给定到对应的寄存器。PWM 模块寄存器就配置好了。

```
18      PTCONbits.PTEN = 0;//禁止PWM模块
19      PTCON = PTCON_INIT;
20      PTCON2 = PTCON2_INIT;
21      //PWM发生器1的寄存器初始化
22      PWMCON1 = PWMCON1_INIT;
23      IOCON1 = IOCON1_INIT;
24      FCLCON1 = FCLCON1_INIT;
25      PHASE1 = PHASE1_INIT;
26      PDC1 = PDC1_INIT;
27      DTR1 = DTR1_INIT;
28      ALTDTR1 = ALTDTR1_INIT;
29      TRGCON1 = TRGCON1_INIT;
30      TRIG1 = TRIG1_INIT;
31      //PWM发生器2的寄存器初始化
32      PWMCON2 = PWMCON2_INIT;
33      IOCON2 = IOCON2_INIT;
34      FCLCON2 = FCLCON2_INIT;
35      PHASE2 = PHASE2_INIT;
36      PDC2 = PDC2_INIT;
37      DTR2 = DTR2_INIT;
38      ALTDTR2 = ALTDTR2_INIT;
39      //PWM发生器3的寄存器初始化
40      PWMCON3 = PWMCON3_INIT;
41      IOCON3 = IOCON3_INIT;
42      FCLCON3 = FCLCON3_INIT;
43      PHASE3 = PHASE3_INIT;
44      PDC3 = PDC3_INIT;
45      DTR3 = DTR3_INIT;
46      ALTDTR3 = ALTDTR3_INIT;
47      PTCONbits.PTEN = 1;//使能PWM模块
48
49  }
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

最后，不要忘记把 PWM 模块打开。大家把今天的内容消化一下，自己下去一定要动手练习，一定要能测试到实际波形。下次课，我们开始介绍 ADC 模块啊。

今天内容就到这里，大家晚安！谢谢大家！

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。